



国家能源集团
CHN ENERGY

国能信控
CHN ENERGY I&C

物联网平台接入标准

物联网平台接入标准

1. 项目概述
 - 1.1 项目名称及用途
 - 1.2 对接方式选择
2. 现场必备条件
 - 2.1 网络环境要求
 - 2.2 现场配置信息清单
3. 网络拓扑与连接信息
 - 3.1 网络架构图
 - 3.2 协议适配对照表
 - 3.3 连接配置参数（按协议类型）
 - 3.3.1 MQTT协议配置
 - 3.3.2 TCP/UDP协议配置
 - 3.3.3 HTTP协议配置
 - 3.3.4 WebSocket协议配置
 - 3.3.5 Kafka协议配置
 - 3.3.6 Modbus-TCP协议配置
4. 产品定义
 - 4.1 产品编码
 - 4.2 设备编码规则
5. 数据通信协议
 - 5.1 协议标准
 - 5.2 协议适配说明
 - 5.3 MQTT协议Topic定义（方式一/标准协议）
 - 5.3.1 上行Topic（合作方 → 物联平台）
 - 5.3.2 下行Topic（物联平台 → 合作方）
 - 5.4 非MQTT协议适配要点（方式二/方式三）
6. 报文格式规范
 - 6.1 通用请求报文结构（MQTT/HTTP/WebSocket/Kafka）
 - 6.2 通用响应报文结构
 - 6.3 TCP/UDP自定义报文格式（示例）
 - 6.4 Modbus-TCP报文映射
7. 数据点表（属性定义）
 - 7.1 Modbus-TCP寄存器映射（如适用）
8. 数据上报（上行）
 - 8.1 MQTT协议上报
 - 8.1.1 单属性上报
 - 8.1.2 多属性批量上报
 - 8.2 TCP/UDP/HTTP/WebSocket/Kafka上报
 - 8.3 上报机制配置
9. 事件上报
 - 9.1 MQTT协议事件上报

- 9.2 其他协议事件上报
- 10. 平台下行控制
 - 10.1 MQTT协议下行控制
 - 10.1.1 属性设置（平台 → 合作方）
 - 10.1.2 属性获取（平台 → 合作方）
 - 10.1.3 功能执行（平台 → 合作方）
 - 10.2 HTTP/WebSocket/Kafka下行控制
- 11. OTA固件升级
 - 11.1 升级流程
 - 11.1.1 物联管理平台查询设备版本号
 - 11.1.2 设备上报版本号
 - 11.1.3 物联平台推送升级包信息
 - 11.1.4 设备下载升级包
 - 11.1.5 查看升级进度和结果
- 12. 心跳检测机制
 - 12.1 MQTT协议心跳
 - 12.2 其他协议心跳机制
 - 12.3 心跳配置
- 13. HTTP接口方式（备选/补充）
 - 13.1 设备功能执行接口
 - 13.2 签名认证机制（如需要）
- 14. 实施 checklist
 - 14.1 前置条件确认
 - 14.2 平台侧配置
 - 14.3 合作方开发
 - 14.4 测试验证
- 15. 附录
 - 15.1 错误码参考
 - 15.2 三种对接方式对比速查
 - 15.3 协议选择建议
 - 15.4 术语对照表

1. 项目概述

1.1 项目名称及用途

简要说明本次接入的目的和业务场景。

示例： 本项目旨在将现场部署的【设备名称/三方平台名称】接入物联网平台。

业务目标：

1. 实现设备运行状态的实时监控与数据采集
2. 支持设备属性的远程配置与下发
3. 实现设备功能的远程调用与控制
4. 建立设备在线状态的心跳检测机制

1.2 对接方式选择

根据现场实际情况，本项目采用以下对接方式：

对接方式	适用场景	责任方	关键要求
方式一：三方平台主动适配	三方平台具备开发能力，愿意遵循物联平台标准	三方平台	按照物联平台MQTT协议规范开发对接
方式二：物联平台适配三方平台	三方平台已有成熟协议，不便修改	物联平台	三方平台提供对接协议+技术对接文档
方式三：设备直连接入	设备厂家支持直接对接	设备厂家	设备厂家提供技术接入文档

本项目选择：【方式一/方式二/方式三】

物联平台标准协议：MQTT

物联平台可适配协议：TCP、UDP、WebSocket、HTTP、MQTT、Kafka、Modbus-TCP 等

2. 现场必备条件

2.1 网络环境要求

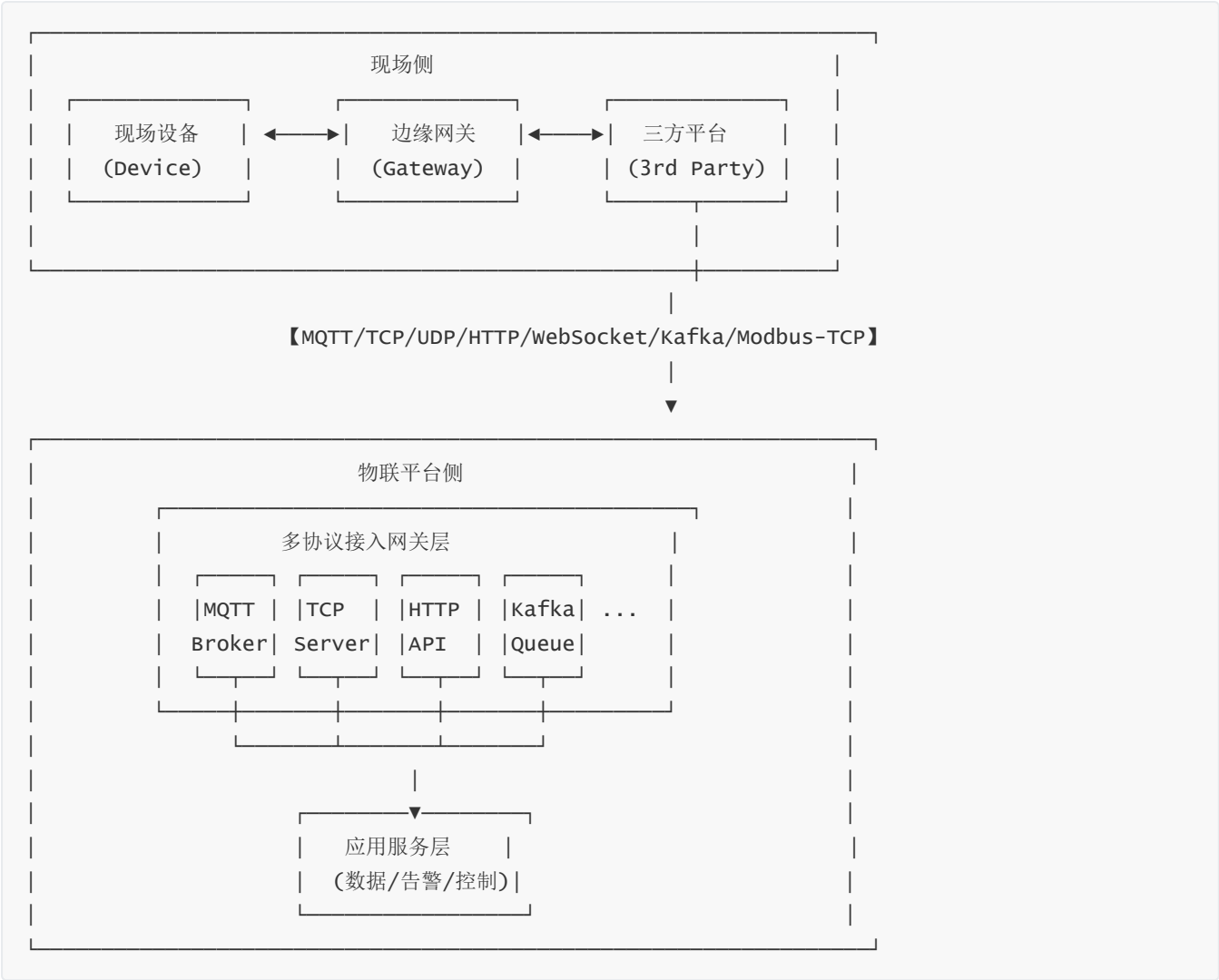
条件	要求	状态
网络打通	设备/网关与物联平台网络双向连通	☐ 待确认
出站端口	开放对应协议端口	☐ 待确认
防火墙策略	允许长连接（TCP/WebSocket/MQTT等）	☐ 待确认

2.2 现场配置信息清单

信息项	内容	提供方
设备/平台IP地址	【填写】	客户/厂家
设备/平台端口号	【填写，如：1883】	客户/厂家
协议类型	【MQTT/TCP/UDP/HTTP/WebSocket/Kafka/Modbus-TCP等】	双方确认
网关出口IP（如有）	【填写】	客户
平台服务器IP	【填写】	物联平台
平台接入地址	【如：mqtt://xxx.xxx.com:1883】	物联平台
认证信息	用户名/密码/证书/Token/API Key等	物联平台

3. 网络拓扑与连接信息

3.1 网络架构图



3.2 协议适配对照表

协议类型	适用场景	标准端口	连接模式	特点
MQTT	物联网设备首选，低带宽、不稳定网络	1883/8883	发布/订阅	轻量、支持离线消息、QoS保障
TCP	自定义协议、长连接场景	自定义	长连接	灵活、需自定义报文格式
UDP	实时性要求高、可容忍丢包场景	自定义	无连接	低延迟、不保证送达
HTTP	Web服务、API调用、低频上报	80/443	请求/响应	通用、易调试、无状态
WebSocket	实时双向通信、Web端应用	80/443(ws/wss)	全双工长连接	基于HTTP握手、支持双向推送

协议类型	适用场景	标准端口	连接模式	特点
Kafka	大数据量、高吞吐、流式处理	9092	发布/订阅	高可用、持久化、批量处理
Modbus-TCP	工业设备、PLC、传感器直连	502	主从模式	工业标准、简单可靠

3.3 连接配置参数（按协议类型）

3.3.1 MQTT协议配置

配置项	参数值	说明
协议类型	MQTT 3.1/3.1.1/5.0	发布/订阅模式
平台接入地址	mqtt://xxx.xxx.com:1883	标准TCP端口
加密接入地址	mqtt://xxx.xxx.com:8883	TLS加密端口（生产环境推荐）
认证方式	【用户名密码 / 证书 / Token】	根据平台分配
ClientID	【设备唯一标识】	建议：productCode_deviceCode
KeepAlive	60秒	心跳保活间隔
Clean Session	false	支持离线消息保留
QoS级别	1	至少送达一次

3.3.2 TCP/UDP协议配置

配置项	参数值	说明
协议类型	TCP / UDP	自定义协议
平台接入地址	【填写，如：tcp://xxx.xxx.com:8080】	平台TCP/UDP服务器地址
报文格式	【自定义二进制/JSON/Protobuf等】	双方约定
心跳机制	【如：60秒发送心跳包】	保活检测
重连策略	【如：断线后5秒重连，最多10次】	可靠性保障

3.3.3 HTTP协议配置

配置项	参数值	说明
协议类型	HTTP 1.1 / HTTP 2.0 / HTTPS	请求/响应模式
平台接入地址	【如： https://api.xxx.com/iot 】	RESTful API地址
认证方式	【Token / API Key / OAuth2.0】	请求头携带

配置项	参数值	说明
Content-Type	application/json	数据格式
请求方法	POST / GET / PUT / DELETE	根据接口定义
超时时间	【如：30秒】	请求超时

3.3.4 WebSocket协议配置

配置项	参数值	说明
协议类型	WebSocket / WebSocket Secure	全双工通信
平台接入地址	【如：wss://ws.xxx.com/iot】	WebSocket端点
子协议	【mqtt / json / 自定义】	Sec-WebSocket-Protocol
心跳机制	【如：Ping/Pong帧 30秒】	WebSocket原生保活
认证方式	【URL参数 / Cookie / Header】	连接时携带

3.3.5 Kafka协议配置

配置项	参数值	说明
协议类型	Kafka Producer / Consumer	发布/订阅模式
Bootstrap Servers	【如：kafka.xxx.com:9092】	Kafka集群地址
Topic（上行）	【如：device-data-up】	设备数据上报Topic
Topic（下行）	【如：device-cmd-down】	平台指令下发Topic
Consumer Group	【合作方code】	消费组标识
Acks	all	最高可靠性
认证方式	【SASL/PLAIN / SSL】	安全认证

3.3.6 Modbus-TCP协议配置

配置项	参数值	说明
协议类型	Modbus-TCP	工业标准协议
平台/网关地址	【如：192.168.1.100:502】	Modbus网关IP
Slave ID	【如：1】	从站地址
功能码	【03/04/06/16等】	读/写寄存器
寄存器映射	【见数据点表】	地址与属性对应关系

配置项	参数值	说明
轮询周期	【如：5秒】	采集频率

4. 产品定义

4.1 产品编码

参数	值	说明
productCode（产品编码）	【填写】	平台侧定义的产品类型标识

4.2 设备编码规则

参数	值	说明
deviceCode（设备编码）	【填写】	设备唯一标识，建议采用SN序列号
编码示例	SN202503130001	项目缩写+日期+序号

5. 数据通信协议

5.1 协议标准

本次接入协议：【MQTT / TCP / UDP / HTTP / WebSocket / Kafka / Modbus-TCP】

通信模式：【发布/订阅 / 请求响应 / 长连接 / 主从模式】

数据格式：【JSON / 二进制 / Protobuf / 自定义】

编码格式：【UTF-8 / GBK / 自定义】

5.2 协议适配说明

方式一（三方平台主动适配）： 需按照物联平台MQTT标准协议对接，详见5.3节
方式二（物联平台适配三方）： 物联平台支持适配 TCP/UDP/WebSocket/HTTP/MQTT/Kafka/Modbus-TCP 等协议，三方平台需提供： - 协议规范文档（报文格式、字段定义、交互流程） - 接口定义文档（API列表、请求响应示例） - 测试环境或模拟器
方式三（设备直连接入）： 物联平台支持适配 TCP/UDP/HTTP/MQTT/Kafka/Modbus-TCP 等协议，设备厂家需提供： - 设备通信协议文档 - 寄存器地址表（Modbus-TCP场景） - SDK或开发指南（如有）

5.3 MQTT协议Topic定义（方式一/标准协议）

5.3.1 上行Topic（合作方 → 物联平台）

用途	Topic格式	说明
数据上报	{productCode}/{deviceCode}/up/report	设备属性数据上报
事件上报	{productCode}/{deviceCode}/up/report/event/{eventCode}	设备告警/事件上报
功能执行回复	{productCode}/{deviceCode}/down/func/{functionCode}/sync/reply	同步功能执行结果回复
心跳回复	{productCode}/{deviceCode}/down/heartbeat/reply	心跳检测响应

5.3.2 下行Topic（物联平台 → 合作方）

用途	Topic格式	说明
属性设置	{productCode}/{deviceCode}/{propertyCode}/down/set	设置设备属性值
属性获取	{productCode}/{deviceCode}/{propertyCode}/down/get	查询设备属性值
功能执行（单设备）	{productCode}/{deviceCode}/down/func/{functionCode}/{sync}	sync: sync(同步)/async(异步)
功能执行（产品级）	{productCode}/down/func/{functionCode}/{sync}	批量执行产品下所有设备
心跳检测	{productCode}/{deviceCode}/down/heartbeat	平台发起心跳检测

通配符订阅说明：合作方可使用 + 通配符订阅某一设备的所有属性，例如： {productCode}/{deviceCode}/+/down/set

5.4 非MQTT协议适配要点（方式二/方式三）

协议	适配要点	需要文档
TCP/UDP	自定义报文解析、粘包处理、心跳保活	报文格式定义、字段说明、交互流程
HTTP	RESTful API映射、认证方式、重试机制	API接口文档、认证说明、错误码定义
WebSocket	连接管理、消息路由、子协议处理	消息格式、事件定义、连接规范
Kafka	Topic映射、分区策略、消费组管理	Topic定义、消息格式、消费策略
Modbus-TCP	寄存器映射、功能码转换、轮询策略	寄存器地址表、功能码说明、数据格式

6. 报文格式规范

6.1 通用请求报文结构（MQTT/HTTP/WebSocket/Kafka）

所有请求报文需包含以下标准头：

```
{
  "requestId": "30dea5ab923641908edbe84e03e3e1ec",
  "requestAppCode": "【合作方code】",
  "targetAppCode": "【目标系统码，如：iot-message-bus】",
  "busiCode": "【业务编码】",
  "signCode": "【验签码，HTTP/Kafka方式必填】",
  "requestTime": "20250313140600",
  "data": {
    // 具体业务数据
  }
}
```

参数名	是否必传	类型	描述
requestId	Y	String	本次请求唯一ID（UUID）
requestAppCode	Y	String	发起请求系统码（合作方code）
targetAppCode	Y	String	被请求系统码
busiCode	Y	String	业务编码
signCode	N	String	验签码（队列方式使用）
requestTime	Y	String	请求时间（yyyyMMddHHmmss）
data	Y	Object	请求体数据

6.2 通用响应报文结构

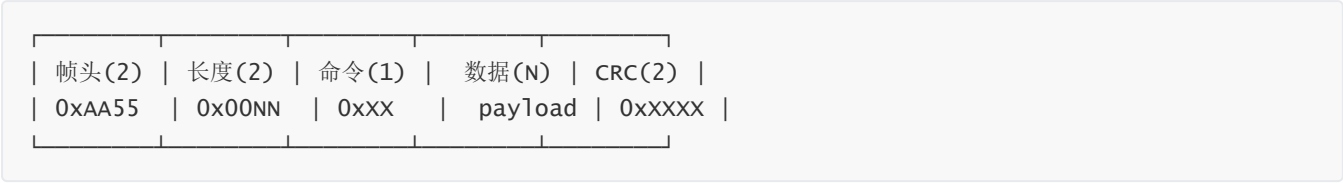
```
{
  "requestId": "30dea5ab923641908edbe84e03e3e1ec",
  "requestAppCode": "【原请求方系统码（合作方code）】",
  "returnAppCode": "【响应方系统码】",
  "responseTime": "20250313140615",
  "returnCode": 200,
  "returnMsg": "成功",
  "data": {}
}
```

参数名	是否必传	类型	描述
requestId	Y	String	原请求ID
requestAppCode	N	String	发起请求系统码（合作方code）

参数名	是否必传	类型	描述
returnAppCode	N	String	返回响应的系统码
responseTime	Y	String	响应时间（yyyyMMddHHmmss）
returnCode	Y	Int	200：成功，其他：失败
returnMsg	Y	String	返回处理消息
data	Y	Object	返回数据

6.3 TCP/UDP自定义报文格式（示例）

如采用TCP/UDP自定义协议，报文格式如下：



字段	长度	说明
帧头	2字节	固定0xAA55，标识帧起始
长度	2字节	后续数据长度（命令+数据+CRC）
命令	1字节	业务命令码
数据	N字节	业务数据（JSON/二进制）
CRC	2字节	校验码

6.4 Modbus-TCP报文映射

Modbus功能码	操作	物联平台映射	说明
0x01	读线圈	读开关量属性	布尔值
0x02	读离散输入	读开关量属性	布尔值
0x03	读保持寄存器	读数值属性	16位/32位整数
0x04	读输入寄存器	读数值属性	16位/32位整数
0x05	写单线圈	写开关量属性	布尔值
0x06	写单寄存器	写数值属性	16位整数
0x0F	写多线圈	批量写开关量	布尔值数组
0x10	写多寄存器	批量写数值	16位整数数组

7. 数据点表（属性定义）

属性编码 (propertyCode)	属性名称	数据类型	单位	读写类型	上报频率	量程/取值范围	小数精度	说明
【如：temp_01】	【温度1】	String/Number	℃	R/W	5分钟	-40~150	2	设备温度
【如：switch_status】	【开关状态】	String	-	R/W	变化上报	0/1	0	0=关, 1=开
【如：energy_total】	【累计能耗】	String	kWh	R	15分钟	0~999999	2	总用电量
【填写】	【填写】	【填写】	【填写】	【填写】	【填写】	【填写】	【填写】	【填写】

读写类型说明：R=只读（设备上报），W=只写（平台下发），R/W=可读写

7.1 Modbus-TCP寄存器映射（如适用）

属性编码	寄存器类型	寄存器地址	功能码	数据类型	系数	单位
temp_01	保持寄存器	40001	0x03	16位有符号	0.1	℃
switch_status	线圈	00001	0x01	布尔	1	-
energy_total	保持寄存器	40010-40011	0x03	32位无符号	1	kWh

8. 数据上报（上行）

8.1 MQTT协议上报

8.1.1 单属性上报

Topic: {productCode}/{deviceCode}/up/report

```
{
  "requestId": "123412",
  "requestAppCode": "【合作方code】",
  "targetAppCode": "iot-message-bus",
  "busiCode": "reportDeviceData",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": {
    "code": "temp_01",
    "value": "24.71",
    "times": "2026-07-21 11:03:56"
  }
}
```

8.1.2 多属性批量上报

Topic: {productCode}/{deviceCode}/up/report

```
{
  "requestId": "123412",
  "requestAppCode": "【合作方code】",
  "targetAppCode": "iot-message-bus",
  "busiCode": "reportDeviceData",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": [
    {
      "code": "temp_01",
      "value": "24.71",
      "times": "2026-07-21 11:03:56"
    },
    {
      "code": "switch_status",
      "value": "1",
      "times": "2026-07-21 11:03:56"
    }
  ]
}
```

8.2 TCP/UDP/HTTP/WebSocket/Kafka上报

协议	上报方式	地址/Topic	数据格式
TCP	建立连接后发送	【ip:port】	自定义报文/JSON
UDP	直接发送数据包	【ip:port】	自定义报文/JSON
HTTP	POST请求	【 https://api.xxx.com/data 】	JSON
WebSocket	send消息	【wss://ws.xxx.com/iot】	JSON
Kafka	Producer发送	【Topic: device-data-up】	JSON

8.3 上报机制配置

配置项	配置值	说明
采集频率	【如：60秒】	设备/网关轮询采集间隔
上报频率	【如：5分钟】	向平台批量上报间隔
变化上报	【是/否】	是否支持变化量触发上报
变化阈值	【如：>5%或绝对值>0.5】	变化量判定阈值
存储周期	【如：3个月】	平台侧数据保留时长

配置项	配置值	说明
压缩方式	【如：Gzip / 不压缩】	大数据量时启用

9. 事件上报

9.1 MQTT协议事件上报

Topic: {productCode}/{deviceCode}/up/report/event/{eventCode}

```
{
  "requestId": "123412",
  "requestAppCode": "【合作方code】",
  "targetAppCode": "iot-message-bus",
  "busiCode": "reportDeviceData",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": {
    "deviceCode": "SN202503130001",
    "code": "high_temp_alarm",
    "value": "[{\"alarmLevel\":\"high\",\"currentTemp\":\"85.5\"}]",
    "time": "2026-07-21 11:03:56",
    "extra": "设备温度过高告警"
  }
}
```

9.2 其他协议事件上报

协议	事件触发方式	上报机制
TCP/UDP	异常检测后主动发送	立即上报，可缓存
HTTP	POST请求	实时上报
WebSocket	emit事件	实时推送
Kafka	Producer发送	实时入队

参数名	是否必传	类型	描述
deviceCode	Y	String	设备编码
code	Y	String	事件编码
value	Y	String	事件内容（JSON字符串）
time	N	Long	毫秒级时间戳
extra	N	String	额外参数/备注

10. 平台下行控制

10.1 MQTT协议下行控制

10.1.1 属性设置（平台 → 合作方）

Topic: `{productCode}/{deviceCode}/{propertyCode}/down/set`

平台下发示例:

```
{
  "requestId": "2132141241",
  "requestAppCode": "iot-device",
  "targetAppCode": "【合作方code】",
  "busiCode": "setDeviceProperty",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": {
    "value": "24.5",
    "times": "2026-07-21 11:03:56"
  }
}
```

合作方响应: 设置成功后, 需主动上报更新后的属性值 (见8.1)

10.1.2 属性获取（平台 → 合作方）

Topic: `{productCode}/{deviceCode}/{propertyCode}/down/get`

机制说明: 平台下发获取请求后, 合作方立即上报当前最新属性值 (见8.1)

10.1.3 功能执行（平台 → 合作方）

Topic: `{productCode}/{deviceCode}/down/func/{functionCode}/{sync}`

sync参数: `sync` (同步, 需等待回复) / `async` (异步, 无需等待)

平台下发示例:

```
{
  "requestId": "2132141241",
  "requestAppCode": "iot-device",
  "targetAppCode": "【合作方code】",
  "busiCode": "executeDeviceFunc",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": {}
}
```

合作方同步回复 (仅sync模式):

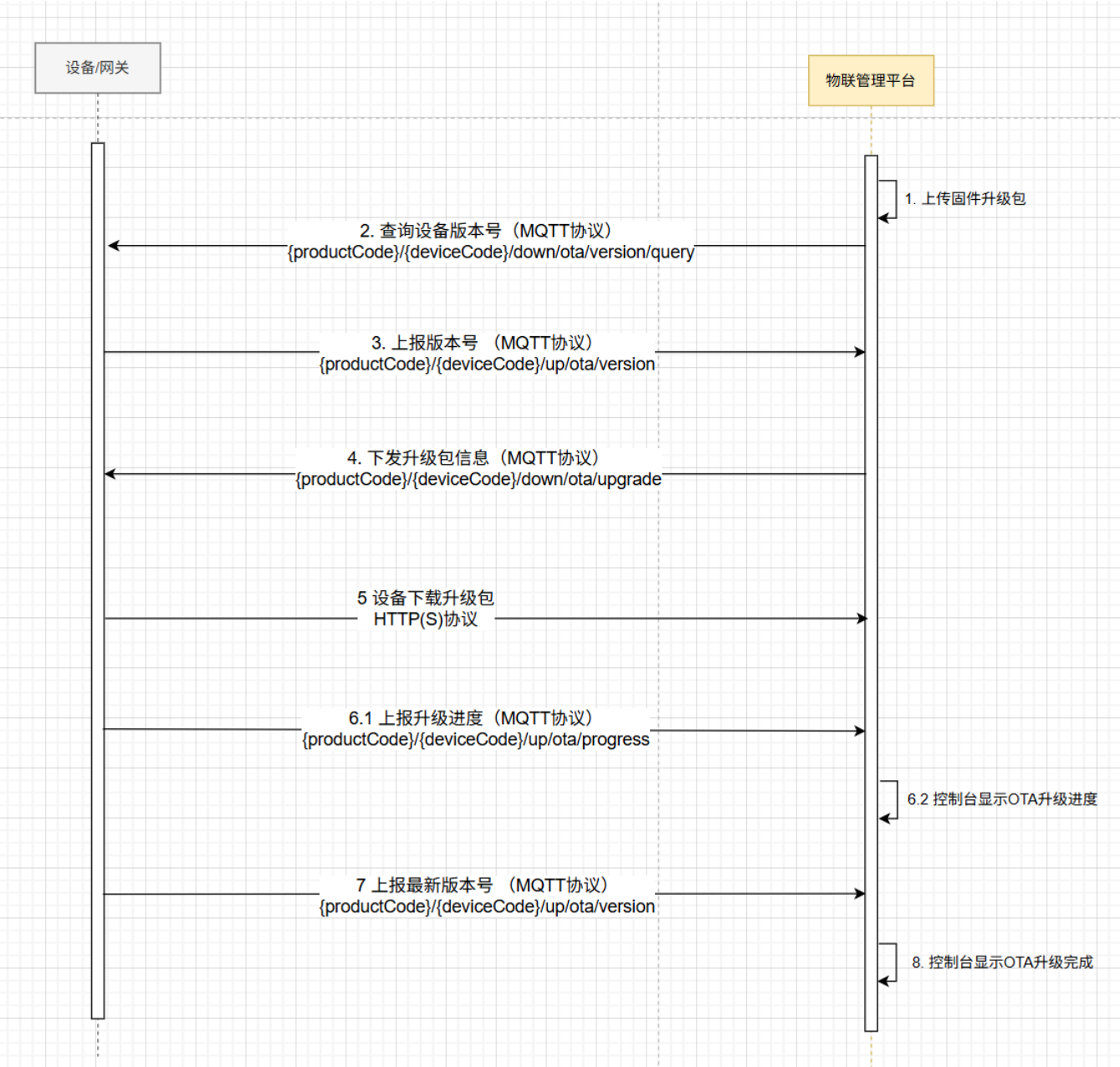
Topic: `{productCode}/{deviceCode}/down/func/{functionCode}/sync/reply`

```
{
  "requestId": "2132141241",
  "requestAppCode": "iot-device",
  "targetAppCode": "【合作方code】",
  "busiCode": "executeDeviceFunc",
  "signCode": "",
  "returnCode": 200,
  "returnMsg": "操作成功",
  "responseTime": "20250313140601",
  "data": {}
}
```

10.2 HTTP/WebSocket/Kafka下行控制

协议	控制方式	机制说明
HTTP	平台调用合作方API	合作方提供回调接口地址
WebSocket	平台emit消息	双向通道，实时下发
Kafka	平台Producer发送	合作方Consumer订阅指令Topic

11. OTA固件升级



11.1 升级流程

11.1.1 物联管理平台查询设备版本号

topic: `{productCode}/{deviceCode}/down/ota/version/query`

物联管控平台主动发布查询设备版本号消息，设备订阅到消息之后，主动上报设备最新版本号。

11.1.2 设备上报版本号

topic: `{productCode}/{deviceCode}/up/ota/version`

设备需要在升级前上报版本号。仅支持每次上报一个模块及对应的模块版本，如果需要上报多个模块的版本，请分次上报。

参数名	是否必传	参数类型	参数描述	默认值
version	Y	String	版本号	
module	Y	String	模块	default
lastUpgradeTime	N	秒级timestamp	上次升级时间	

```
{
  "requestId": "2132141241",
  "requestAppCode": "调用发系统简称",
  "targetAppCode": "iot-device",
  "busiCode": "reportDeviceVersion",
  "signCode": "",
  "requestTime": "20250312103000",
  "data": {
    "version": "1.0.0",
    "module": "default",
    "lastUpgradeTime": 1747897234
  }
}
```

11.1.3 物联平台推送升级包信息

topic: {productCode}/{deviceCode}/down/ota/upgrade

- 1. 物联管理平台上传设备升级固件包，上传至S3存储平台
- 2. 对指定产品/设备发起升级任务：发布升级包消息

参数名	是否必传	参数类型	参数描述	默认值
version	Y	String	版本号	
module	Y	String	模块	default
size	Y	Long	升级包大小，单位：字节	
signMethod	Y	String	签名方式 MD5/SHA256	
sign	Y	String	签名	
url	Y	String	升级包S3存储下载地址（HTTP）	
extra	N	String	额外信息	

```
{
  "requestId": "2132141241",
  "requestAppCode": "iot-device",
  "targetAppCode": "目标方系统简称",
  "busiCode": "otaUpgrade",
  "signCode": "",
  "requestTime": "20250312103000",
}
```

```
"data": {
  "version": "1.0.1",
  "module": "default",
  "size": 60458000
  "signMethod": "MD5",
  "sign": "b008369d78e46a532e11b3e98d19684a"
  "url":
"http://127.0.0.1:9000/iotfile/firmware/14467c9ed0f947d956ff30515e31f024/default/1.0.1/iot-
link-plugin-demo-1.1.jar",
  "extra": "额外信息"
}
}
```

11.1.4 设备下载升级包

1. 设备订阅升级包topic
2. 根据设备自身协议进行OTA升级，可选择HTTP或者MQTT
3. 设备上报升级进度

topic: {productCode}/{deviceCode}/up/ota/progress

参数名	是否必传	参数类型	参数描述	默认值
progress	Y	Integer	OTA升级进度。设备上报的进度值及其描述信息 1~100的整数：升级进度百分比。其他则为升级异常值	
module	Y	String	模块	default
desc	N	String	当前步骤的描述信息，。如果发生异常，此字段可承载错误信息。	

```
{
  "requestId": "2132141241",
  "requestAppCode": "调用发系统简称",
  "targetAppCode": "iot-device",
  "busiCode": "upOtaProgress",
  "signCode": "",
  "requestTime": "20250312103000",
  "data": {
    "progress": 50,
    "module": "default",
    "desc": ""
  }
}
```

4. 升级完成 [设备上报版本号](#)

11.1.5 查看升级进度和结果

在物联管理平台可查看设备升级进度。
只有当设备上报最新的版本号等于目标OTA版本号之后，才算设备OTA升级成功。

12. 心跳检测机制

12.1 MQTT协议心跳

** 心跳请求参数: **

参数名	是否必传	类型	描述
productCode	Y	String	产品编码
data	Y	arry	设备编码集合

心跳请求Topic: {productCode}/down/heartbeat

心跳请求下发:

```
{
  "requestId": "1787620439416",
  "requestAppCode": "iot-device",
  "targetAppCode": "【合作方code】",
  "busiCode": "deviceHeartbeat",
  "signCode": "",
  "returnCode": 200,
  "returnMsg": "操作成功",
  "responseTime": "20250313140601",
  "data": ["deviceCode1","deviceCode2"]
}
```

** 心跳回复参数: **

参数名	是否必传	类型	描述
productCode	Y	String	产品编码
data	Y	Map<string,boolean>	是否在线 返回key为设备编码，value为boolean标识设备是否在线

合作方回复Topic: {productCode}/down/heartbeat/reply

合作方回复:

```
{
  "requestId": "1787620439416",
  "requestAppCode": "iot-device",
  "targetAppCode": "【合作方code】",
  "busiCode": "deviceHeartbeat",
  "signCode": "",
  "returnCode": 200,
  "returnMsg": "操作成功",
  "responseTime": "20250313140601",
  "data": {
    "deviceCode1":true,
    "deviceCode2":false
    .....
  }
}
```

12.2 其他协议心跳机制

协议	心跳机制	配置参数
TCP	自定义心跳包	【如：60秒发送一次，3次无响应断开】
UDP	无连接，不维持心跳	通过数据上报频率判断在线
HTTP	无状态，通过轮询或长轮询	【如：5分钟调用一次状态接口】
WebSocket	Ping/Pong帧	【如：30秒发送Ping】
Kafka	消费者心跳	由Kafka客户端自动维护
Modbus-TCP	轮询读取	【如：5秒读取一次状态寄存器】

12.3 心跳配置

配置项	参数值	说明
检测周期	【如：60秒】	平台心跳检测间隔
超时判定	【如：3次未响应】	设备离线判定阈值
保活机制	合作方建议实现	主动发送心跳维持连接

13. HTTP接口方式（备选/补充）

13.1 设备功能执行接口

URL: `http://{ip}:{port}/device/function/{productCode}/{deviceCode}/{functionCode}`

请求方法: POST

请求头:

- Code: `【iam校验码】`（或采用签名认证）

请求体:

```
{
  "requestId": "2132141241",
  "requestAppCode": "iot-data",
  "targetAppCode": "【调用方系统简称】",
  "busiCode": "executeDeviceFunc",
  "signCode": "",
  "requestTime": "20250313140600",
  "data": {}
}
```

13.2 签名认证机制（如需要）

认证方式: 基于appkey + appsecret的MD5签名

请求头参数:

- X-App-Key: `【appkey】`
- X-Timestamp: `【yyyyMMddHHmmss】`
- X-Sign: `【计算签名】`

签名计算步骤:

1. 准备参数:

- appKey: 平台分配的应用标识
- timestamp: 当前时间戳 (yyyyMMddHHmmss)
- body: 请求体Base64编码后的内容
- appSecret: 平台分配的应用密钥

2. 构建签名字符串:

```
appkey=【appkey】&timestamp=【时间戳】&body=【body内容】&appSecret=【appsecret】
```

3. 计算MD5哈希: 得到最终sign值

14. 实施 checklist

14.1 前置条件确认

序号	检查项	状态	备注
1	对接方式确认（方式一/二/三）	☐	见第1.2节
2	协议类型确认 (MQTT/TCP/UDP/HTTP/WebSocket/Kafka/Modbus-TCP)	☐	见第5.1节
3	技术文档获取	☐	方式二需三方协议文档； 方式三需厂家技术文档
4	网络连通性测试（合作方→平台对应协议端口）	☐	确认端口通
5	现场配置信息收集（IP、端口、协议参数等）	☐	见第2.2节

14.2 平台侧配置

序号	检查项	状态	备注
6	产品编码（productCode）已分配	☐	平台侧配置
7	设备编码（deviceCode）规则确认	☐	建议SN序列号
8	协议接入网关配置（按协议类型）	☐	MQTT Broker/TCP Server/HTTP API等
9	认证信息获取（按协议类型）	☐	用户名/密码/证书/Token/API Key等
10	数据点表（属性编码）与平台对齐	☐	双方确认
11	Topic/接口/寄存器映射配置	☐	平台侧授权
12	合作方code注册	☐	平台侧分配唯一标识

14.3 合作方开发

序号	检查项	状态	备注
13	协议客户端连接实现	☐	按选定协议开发
14	数据上报功能实现	☐	单属性/多属性/批量上报
15	属性设置响应机制实现	☐	收到set后上报新值
16	属性获取响应机制实现	☐	收到get后立即上报
17	功能执行回复机制实现（同步模式）	☐	同步调用需回复
18	心跳检测响应机制实现	☐	按协议类型实现

序号	检查项	状态	备注
19	事件上报功能实现（如有告警）	☐	告警/异常事件上报
20	断线重连机制实现	☐	TCP/MQTT/WebSocket等长连接协议
21	离线缓存机制实现（可选）	☐	网络中断时本地存储

14.4 测试验证

序号	检查项	状态	备注
22	数据上报端到端测试	☐	平台侧确认收到数据
23	属性设置下发测试	☐	平台→合作方→确认生效
24	功能执行调用测试	☐	同步/异步模式验证
25	心跳检测机制测试	☐	在线/离线状态准确
26	断线重连测试	☐	模拟网络中断恢复
27	离线缓存测试（如启用）	☐	网络中断时本地存储
28	压力测试（高频上报场景）	☐	大批量数据稳定性
29	安全认证测试	☐	非法连接拒绝

15. 附录

15.1 错误码参考

returnCode	含义	处理建议
200	成功	正常
400	请求参数错误	检查请求体格式与参数
401	认证失败	检查认证信息
403	权限不足	检查Topic/接口权限配置
404	设备/属性不存在	检查编码是否正确
408	请求超时	检查网络或合作方响应
500	服务器内部错误	联系平台管理员
502	网关错误	协议适配层异常
503	服务不可用	稍后重试
2000	设备操作失败（自定义）	查看returnMsg详情

15.2 三种对接方式对比速查

维度	方式一：三方平台主动适配	方式二：物联平台适配三方	方式三：设备直连接入
前提条件	三方平台具备开发能力	三方平台提供协议文档	设备厂家提供技术文档
协议标准	物联平台MQTT标准	支持TCP/UDP/HTTP/WebSocket/MQTT/Kafka/Modbus-TCP等	支持TCP/UDP/HTTP/MQTT/Kafka/Modbus-TCP等
开发责任	三方平台（合作方）	物联平台	双方协商
对接周期	较短（标准协议）	中等（需适配开发）	视设备复杂度
维护成本	低（标准统一）	中（定制化）	低（单设备定制）
扩展性	高（标准复用）	中（case by case）	低（单设备定制）
适用场景	新建系统/有开发资源	存量系统/不便改造/非MQTT协议	专用设备/无网关/工业协议

15.3 协议选择建议

场景	推荐协议	理由
物联网设备、传感器、低带宽网络	MQTT	轻量、支持离线、QoS保障
自定义协议、二进制数据、高实时性	TCP/UDP	灵活、开销小
Web应用、浏览器端、实时推送	WebSocket	全双工、基于HTTP握手
API集成、低频上报、无状态请求	HTTP	通用、易调试、防火墙友好
大数据量、流式处理、高吞吐	Kafka	高可用、持久化、可回溯
工业设备、PLC、传感器直连	Modbus-TCP	工业标准、简单可靠、广泛支持
混合场景	多协议并存	物联平台支持多协议统一接入

15.4 术语对照表

术语	说明
合作方code	合作方系统唯一标识，对应requestAppCode/returnAppCode
productCode	产品编码，平台侧定义的产品类型标识
deviceCode	设备编码，设备唯一标识
propertyCode	属性编码，设备数据点标识
functionCode	功能编码，设备功能标识

术语	说明
eventCode	事件编码，设备事件/告警标识

文档版本： v1.2
基于规范： 物联平台接入标准 v1.2
更新日期： 2026-05-06 14:06